

Class 14: Structured Data, Encoding, and Embeddings

Data-driven Systems Engineering

Agenda

- From data type to data structure: structured, semi-structured, unstructured.
- Where each structure shows up in this repository's real services.
- Why raw values must be encoded before a model can use them.
- One-hot, ordinal, target, and cyclic encoding.
- Encoding versus embedding as two different compression strategies.
- Dimensionality reduction (PCA) as feature extraction after encoding.

Learning goals

- Classify a dataset as structured, semi-structured, or unstructured, and explain why the distinction matters.
- Choose an encoding strategy that matches a variable's cardinality and meaning.
- Explain the difference between a manually designed encoding and a learned embedding.
- Understand PCA as one instance of a broader idea: compressing a representation once it is numeric.

Course Map and Running Cases



Today in the course

Focus: Structured, semi-structured, and unstructured data, and the encoding and embedding theory that turns any of them into numbers.

Bridge: Class 13 asked what kind of data we have. This class asks how it is organized, and how we turn it into the numeric representation every model actually consumes.

Fraud case

The fraud dataset is structured and already partly encoded: 'V1'-'V28' are anonymized, PCA-derived numeric columns rather than raw transaction attributes.

Pasteurization case

The pasteurization service exchanges semi-structured JSON events over its streaming endpoint, even though the underlying sensor values are simple numbers.

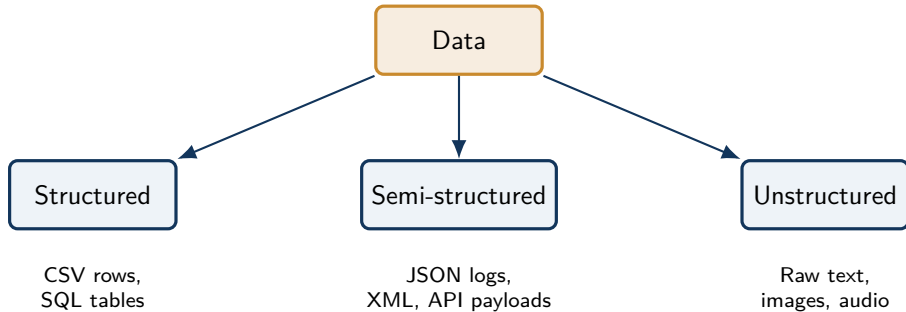
From data type to data structure

- Class 13 asked whether data is tabular, time series, spectral, or image.
- This class asks a orthogonal question: how organized is it, structured, semi-structured, or unstructured?
- Any of the four data types from Class 13 can appear in any of these three organizational forms.
- The organizational form determines how much encoding work is needed before a model can use the data at all.

Structured, semi-structured, and unstructured data

Structured	Fixed schema, fits directly into rows and columns: a CSV file or a SQL table.
Semi-structured	Has organization (keys, tags, nesting) but no fixed column schema: JSON logs, XML documents, API payloads.
Unstructured	No inherent schema at all: raw text, images, and audio waveforms.

The taxonomy as a tree



Semi-structured data in this course's own services

- It is tempting to think of this course as purely tabular, but two real services already speak semi-structured data.
- A Flask endpoint receiving `'request.get_json(force=True)'` is accepting a semi-structured payload before it is ever turned into a feature array.
- The pasteurization streaming endpoint emits one semi-structured JSON event per second, not a table.
- Both are converted into structured, tabular inputs before a model touches them.

Semi-structured example: a streaming JSON event

```
def generate():  
    for _, row in df.iterrows():  
        data = row.to_dict()  
        yield f"data: {json.dumps(data)}\n\n"  
        time.sleep(STREAM_DT)
```

- From 'pasteurization/serving.py': each streamed event is a small, self-describing JSON object with named keys.
- Nothing forces every event to have the same keys, which is exactly what distinguishes semi-structured data from a rigid table.

Unstructured data

- Raw text, images, and audio carry no built-in fields or schema; meaning has to be extracted, not read off a column name.
- Unstructured data almost always requires a learned representation (an embedding) rather than a small set of hand-designed columns.
- None of this repository's two running cases uses unstructured data directly, but production fraud and process-monitoring systems often also ingest free-text notes, images, or logs alongside their tabular core.
- The encoding and embedding ideas in this class scale from “one categorical column” up to “an entire document or image”.

Why encoding exists

- Models compute with numbers; most raw variables (categories, labels, identifiers) are not numbers.
- Encoding is the deliberate, documented choice of how a symbolic value becomes a numeric one.
- A bad encoding can inject a false sense of order, leak information, or blow up dimensionality.
- The right choice depends on cardinality (how many distinct values) and semantics (does order mean anything).

One-hot encoding	Good for low-cardinality categories with no natural order.
Ordinal encoding	Use only when order is real and meaningful.
Target encoding	Useful for high-cardinality categories, but must control leakage.
Cyclic encoding	Useful for hour-of-day, day-of-week, and other seasonal variables.

Encoding example: structured pipeline with one-hot encoding

```
categorical_features = ["pclass", "sex", "embarked", "title"]

categorical_transformer = Pipeline(steps=[
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

- From 'notebooks/Class4_DataEngineering.ipynb': encoding is expressed as an explicit, reusable pipeline step, not an ad hoc conversion.
- 'handle_unknown="ignore"' is itself an encoding decision: what happens when a category appears at inference time that training never saw.

Encoding versus embedding

Encoding A manually designed, usually sparse mapping from symbols to numbers (one-hot, ordinal, target).

Embedding A learned, dense, lower-dimensional vector that a model fits to place similar items close together.

- Encodings are interpretable by construction; embeddings are compact but usually opaque.
- Text embeddings (word2vec-style) and learned image features play the same role for unstructured data that one-hot encoding plays for a categorical column.
- The choice is not ideological: high-cardinality categories and unstructured data push naturally toward embeddings.

Feature extraction: compressing after encoding

- Once every variable is numeric, we can ask a second, different question: can we compress this representation without losing what matters?
- Feature extraction transforms an already-numeric representation into a smaller one, rather than deciding how to encode a single symbol.
- PCA is the classical linear technique for structured, tabular data.
- The same idea has data-type-specific analogues: the Fourier transform or MFCC framing for signals, and learned convolutional or embedding features for images and text.

PCA rotates the original feature space to new orthogonal axes called principal components, ordered by the amount of variance they explain.

- The first component captures the largest possible variance; later components capture what is left, subject to orthogonality.
- Keeping only the leading components compresses the representation while trying to preserve most of the structure.
- PCA must be fit only on training data, then applied identically at inference time.

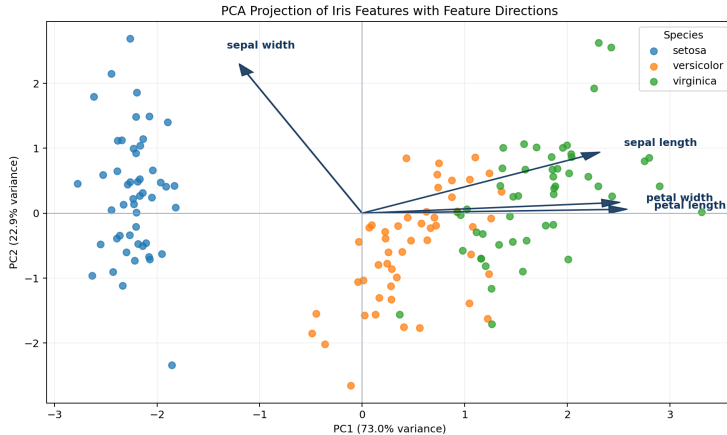
Feature extraction example: PCA after encoding

```
pca_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("pca", PCA(n_components=3)),
    ("classifier", LogisticRegression(max_iter=100))
])

pca_pipeline.fit(X_train, y_train)
y_pred_pca = pca_pipeline.predict(X_test)
```

- From 'notebooks/Class4_DataEngineering.ipynb': PCA is inserted as one more pipeline stage, right after encoding and scaling.
- This is the same ordering principle as the taxonomy in this class: encode first, then decide whether to compress.

PCA projection with feature directions



Recreated from 'notebooks/Class3_PCA.ipynb': once features are numeric, PCA re-expresses them along the directions of largest variance.

Analogues across data types, and the fraud case

Tabular	PCA on encoded, scaled columns.
Signal	Fourier transform or MFCC framing before modeling.
Image / text	Learned convolutional or embedding features.

Fraud example

The fraud dataset already ships with PCA-derived variables 'V1'–'V28'. The encoding and extraction steps happened before the data ever reached this course; the original attributes are permanently hidden.

Repo activity

- 'notebooks/Class4_DataEngineering.ipynb' shows encoding ('OneHotEncoder') and extraction ('PCA') as consecutive pipeline stages.
- 'pasteurization/serving.py' produces the semi-structured JSON stream that a later stage turns into structured features.
- The fraud dataset's 'V1'–'V28' columns are a real example of encoding and extraction decisions made upstream, outside our control.

Question

If the fraud dataset's 'V1'–'V28' are PCA components, what did we lose by never seeing the original raw attributes?

- This connects representation choices back to interpretability, a theme that returns in Class 17's decision-making discussion.

Papers and materials

Zheng and Casari, Feature Engineering for Machine Learning; ColumnTransformer; scikit-learn PCA guide; Jolliffe, Principal Component Analysis; Chip Huyen, Designing Machine Learning Systems.

From This Class to the Next

What stays with us

- Structured, semi-structured, and unstructured data need increasingly deliberate representation work before modeling.
- Encoding is a manual, interpretable mapping; embeddings are learned, dense, and compact.
- Dimensionality reduction such as PCA is feature extraction: compression applied after encoding, not a replacement for it.

Project use

Project teams should now be able to name, for every raw input, whether it needs encoding, an embedding, or both, and whether extraction is worth the interpretability cost.

Next connection

Class 15 uses this numeric representation to clean, curate, select, and construct the actual features a model will train on.